
CAHIER DES CHARGES

Systeme de Stationnement Intelligent

Réalisé par : Koceir Mehdi

Table des matières

1	Étude de Cas	2
2	Description de la Problématique	3
3	Besoins Matériels	3
3.1	Tableau des Composants	4
3.2	Justification du Choix des Composants	5
4	Fonctionnement du Système	5
4.1	Tableau des États et Indications	5
4.2	Logique de Fonctionnement	6
4.3	Exemple d’Affichage LCD	6
5	Simulation	6
5.1	Objectif	6
5.2	Outil Utilisé	6
5.3	Étapes de Simulation	6
6	Réalisation	7
6.1	Étapes de Montage	7
7	Perspectives d’Amélioration	8
8	Conclusion	9

1. Étude de Cas

Contexte : Un parking de taille moyenne reçoit un flux continu de véhicules. Les conducteurs entrent sans connaître l'état réel du parking ni la direction à prendre pour trouver une place libre. Cette situation génère des embouteillages internes, une perte de temps et une circulation inutile à l'intérieur du parking.

Problème 1 : Les conducteurs ignorent s'il reste des places libres avant d'entrer. En l'absence d'un système de comptage, des véhicules s'engagent dans un parking complet, causant des blocages.

Problème 2 : Même lorsque des places sont disponibles, le conducteur ne sait pas dans quelle zone se diriger. Il tourne inutilement entre la Zone A et la Zone B, allongeant la durée d'occupation de l'allée centrale.

Problème 3 : La barrière d'entrée n'est pas automatisée : en l'absence de gardien, aucun contrôle n'est effectué. Un véhicule peut entrer alors que le parking est complet.

Problème 4 — Surcharge du Personnel Humain : La gestion manuelle d'un parking requiert la présence permanente d'agents chargés de guider les conducteurs, de contrôler l'accès, de surveiller les places et de communiquer l'état du parking. Cette organisation entraîne des coûts salariaux élevés, un risque d'erreur humaine (mauvais comptage, inattention), et une absence de continuité de service en dehors des heures ouvrables. Un seul agent ne peut pas simultanément surveiller plusieurs zones, gérer la barrière et informer les conducteurs en temps réel.

Solution proposée : Concevoir un système embarqué basé sur l'ESP32 et 4 capteurs HC-SR04 capables de :

- Compter les entrées et sorties de véhicules en temps réel, **sans intervention humaine**
- Vérifier automatiquement s'il reste des places disponibles
- Ouvrir ou fermer la barrière selon l'état du parking, **remplaçant le rôle du gardien**
- Guider le conducteur vers la Zone A (droite) ou la Zone B (gauche) selon la

disponibilité réelle, **sans qu'un agent soit nécessaire**

- Fonctionner **24h/24 et 7j/7** sans fatigue ni interruption

2. Description de la Problématique

Le parking est divisé en deux zones distinctes : la **Zone A** (droite) et la **Zone B** (gauche). Chaque zone est surveillée par un capteur ultrasonique dédié. Un capteur à l'entrée détecte les arrivées et un capteur à la sortie enregistre les départs. L'ESP32 traite ces informations en temps réel pour maintenir un compteur de places fiable.

Objectifs fonctionnels du système :

- Compter les voitures présentes dans le parking à tout moment
- Vérifier en permanence la condition : `places_libres > 0`
- Ouvrir la barrière automatiquement si une place est disponible
- Afficher sur l'écran LCD la direction vers la zone disponible
- Signaler l'état du parking via les LEDs bleue (libre) et rouge (plein)
- Réduire au minimum l'intervention humaine dans la gestion quotidienne du parking

3. Besoins Matériels

3.1. Tableau des Composants

Illustration	Composant	Définition / Rôle	Quantité / Spec.
	ESP32	Microcontrôleur principal. Traite les données des 4 capteurs, commande le servo, les LEDs et l'afficheur. Intègre Wi-Fi et Bluetooth.	1 unité
	HC-SR04	Capteur ultrasonique. Mesure la distance par émission/réception d'ultrasons. Utilisé à l'entrée, à la sortie, en Zone A et Zone B.	4 unités
	Servo Moteur	Actionne la barrière d'entrée. S'ouvre si une place est disponible, reste fermée si le parking est plein.	1 — SG90, 5V
	LED Bleue	Signal visuel : places disponibles dans le parking (<code>places_libres > 0</code>).	1 — 5 mm, 2V, 20 mA
	LED Rouge	Signal visuel : parking complet, aucune place disponible. Barrière maintenue fermée.	1 — 5 mm, 2V, 20 mA
	Écran LCD / OLED	Affiche le nombre de places libres et la direction à prendre (ZONE A ou ZONE B).	1 — I2C, 16×2 ou 128×64
	Résistances	Protection des LEDs contre les surintensités. Limitent le courant à une valeur sûre.	2 × 220 Ω
	Breadboard + Câbles	Support de prototypage sans soudure. Câbles mâle-mâle et mâle-femelle pour toutes les connexions.	1 kit

3.2. Justification du Choix des Composants

Le choix des composants principaux repose sur des critères de coût, de disponibilité, de compatibilité et d'adéquation fonctionnelle au projet.

ESP32 : L'ESP32 est le microcontrôleur central du système. Il a été retenu pour son très faible coût (environ 3 \$) par rapport à ses capacités : il dispose de suffisamment de GPIO pour connecter simultanément 4 capteurs ultrasoniques, un servo-moteur, deux LEDs et un écran LCD. Sa fréquence d'horloge élevée lui permet de traiter les signaux de tous les capteurs en temps réel sans latence perceptible. De plus, il intègre nativement le Wi-Fi et le Bluetooth, ce qui ouvre la voie à une supervision à distance sans matériel supplémentaire. Aucun autre microcontrôleur dans sa gamme de prix ne réunit autant de fonctionnalités.

HC-SR04 : Le capteur ultrasonique HC-SR04 a été choisi pour détecter la présence de véhicules à l'entrée, à la sortie et dans chacune des deux zones. Il est bon marché (moins de 1 \$), très répandu dans les projets embarqués éducatifs, et ne nécessite que 4 broches (VCC, GND, TRIG, ECHO). Sa plage de mesure, de 2 cm à 4 m, est parfaitement adaptée aux distances rencontrées dans un parking de taille réduite. Sa compatibilité 3,3V/5V le rend directement utilisable avec l'ESP32 sans circuit d'adaptation.

Écran LCD 16×2 avec module I2C : L'afficheur LCD 16×2 a été sélectionné pour communiquer au conducteur les informations essentielles : le nombre de places restantes et la direction à emprunter. Le module I2C intégré réduit le câblage à seulement 2 fils de données (SDA et SCL) au lieu des 6 à 8 fils requis en mode parallèle, ce qui préserve les GPIO disponibles sur l'ESP32. Son affichage textuel est lisible à 1 à 2 mètres de distance, ce qui le rend adapté à une signalétique d'entrée de parking.

4. Fonctionnement du Système

4.1. Tableau des États et Indications

↳Condition	↳LED	↳Barrière	↳Affichage LCD
Places dispo + Zone B libre	Bleue ON	Ouverte	Free: N GO LEFT <- ZONE B
Places dispo + Zone A libre	Bleue ON	Ouverte	Free: N GO RIGHT -> ZONE A
Parking complet (0 place)	Rouge ON	Fermée	PARKING FULL

4.2. Logique de Fonctionnement

Le microcontrôleur exécute en permanence la boucle suivante :

1. Envoyer une impulsion aux 4 capteurs HC-SR04
2. Mesurer la distance renvoyée par chaque capteur
3. Mettre à jour le compteur : incrémenter à l'entrée, décrémenter à la sortie
4. Vérifier la condition : `places_libres > 0`
5. Si disponible : ouvrir le servo (barrière), allumer LED bleue
6. Identifier quelle zone (A ou B) contient des places libres
7. Afficher la direction sur l'écran LCD (`GO RIGHT` vers Zone A ou `GO LEFT` vers Zone B)
8. Si parking plein : fermer barrière, allumer LED rouge, afficher `PARKING FULL`
9. Répéter toutes les 100 ms

4.3. Exemple d’Affichage LCD

```
SMART PARKING
Free: 1      GO RIGHT -> ZONE A
```

5. Simulation

5.1. Objectif

La simulation précède obligatoirement la réalisation physique. Elle permet de valider la logique de guidage, les conditions d'ouverture de la barrière, l'état des LEDs et l'affichage LCD sans risque pour le matériel.

5.2. Outil Utilisé

- **Plateforme** : Wokwi — simulateur compatible avec ESP32
- **Langage** : C++ compilé via Arduino IDE

5.3. Étapes de Simulation

1. Créer le circuit virtuel : ESP32, 4 × HC-SR04, servo, 2 LEDs avec résistances, écran LCD
2. Saisir le code source Arduino dans l'éditeur Wokwi
3. Simuler l'arrivée d'une voiture (Capteur 1 détecte un objet)
4. Tester les scénarios : Zone A libre, Zone B libre, parking complet

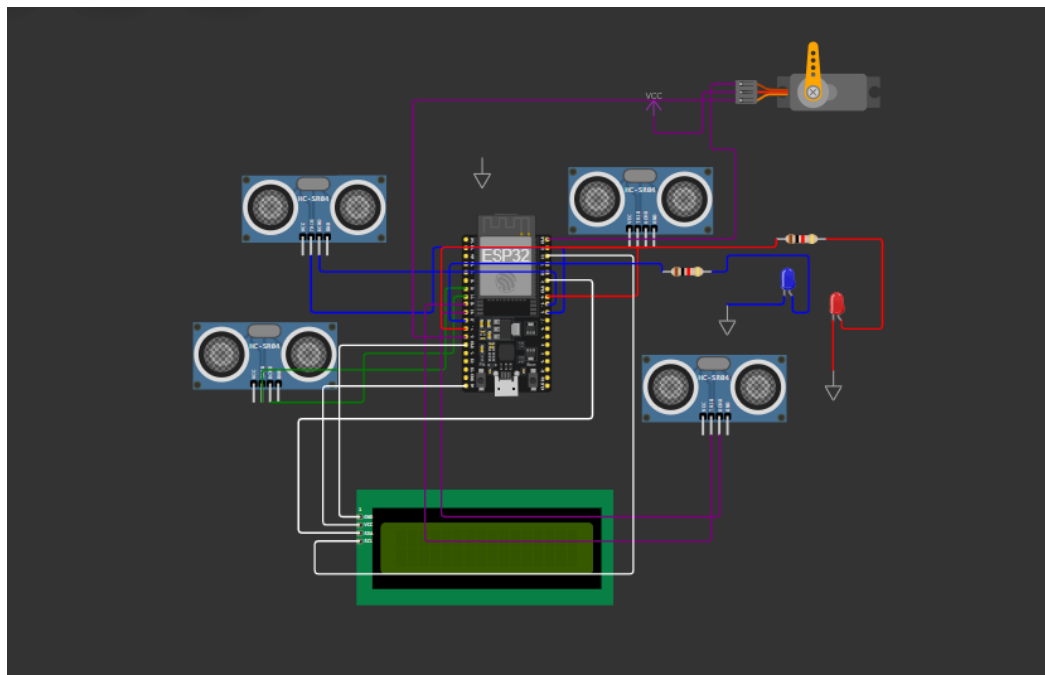


FIGURE 1 – Résultats de la simulation

5. Vérifier la conformité des sorties avec le tableau des états
6. Documenter les résultats par captures d'écran pour le rapport

6. Réalisation

6.1. Étapes de Montage

1. **Préparation** : Rassembler tous les composants listés dans le tableau des besoins matériels
2. **Alimentation** : Connecter l'ESP32 via câble USB à l'ordinateur
3. **Capteurs** : Câbler les $4 \times$ HC-SR04 selon le brochage défini (VCC, GND, TRIG, ECHO)
4. **Servo** : Connecter le signal PWM au GPIO 26, VCC au 5V et GND au GND
5. **LEDs** : Brancher chaque LED en série avec une résistance 220Ω vers le GPIO correspondant
6. **Écran LCD** : Connecter SDA et SCL aux GPIO 21 et GPIO 22 (communication I2C)
7. **Téléversement** : Charger le code validé en simulation via l'Arduino IDE
8. **Test final** : Simuler des entrées et sorties de véhicules et vérifier chaque état

7. Perspectives d'Amélioration

Les améliorations futures envisageables pour ce système dépendent directement de l'architecture physique du parking dans lequel il serait déployé. En effet, la structure, la taille et la disposition des zones conditionnent les évolutions possibles.

Dans un parking de petite taille à architecture linéaire (comme celui modélisé dans ce projet), le système actuel à deux zones est suffisant. Une première amélioration consisterait à ajouter une interface Web ou une application mobile connectée via le Wi-Fi de l'ESP32, permettant aux conducteurs de consulter la disponibilité des places à distance, avant même d'entrer dans le parking.

Pour un parking de taille moyenne à architecture en peigne ou en épi, le système pourrait être étendu à un plus grand nombre de zones, chacune surveillée par un capteur HC-SR04 supplémentaire. L'ESP32 étant capable de gérer plusieurs GPIO simultanément, cette extension ne nécessiterait pas de changer le microcontrôleur, mais simplement d'adapter le code et le câblage. L'affichage LCD pourrait alors être remplacé par un écran OLED ou une signalétique lumineuse en hauteur, plus visible dans de grandes allées.

Dans le cas d'un parking multi-niveaux ou de grande capacité, une architecture distribuée serait nécessaire : plusieurs modules ESP32 communiqueraient entre eux via Wi-Fi ou un protocole MQTT pour centraliser les informations de chaque niveau. Un serveur central agrégerait les données et enverrait les instructions à chaque barrière et afficheur. Des algorithmes d'optimisation pourraient alors orienter les conducteurs vers le niveau ou la zone avec le temps de marche le plus court, et non simplement vers la première place disponible.

Enfin, quelle que soit l'architecture du parking, une amélioration universelle serait l'intégration de capteurs de place individuels (un HC-SR04 par emplacement), permettant de passer d'un système de comptage global à une détection place par place, offrant un guidage encore plus précis et une meilleure gestion de l'espace disponible.

8. Conclusion

Ce cahier des charges définit de manière complète et structurée l'ensemble des exigences techniques, matérielles et fonctionnelles du système de stationnement intelligent basé sur l'ESP32. Par rapport à un simple détecteur de proximité, ce système apporte une valeur ajoutée significative : le **guidage directionnel**, la **gestion automatique de la barrière**, un **affichage en temps réel** et surtout la **réduction drastique de l'intervention humaine**, permettant une gestion efficace du parking sans personnel dédié.

La démarche adoptée — **simulation avant réalisation** — garantit une validation rigoureuse de chaque composant avant le montage physique final. Ce projet démontre comment un microcontrôleur économique comme l'ESP32 peut être intégré dans une application concrète de gestion intelligente d'infrastructure, en combinant acquisition de données multi-capteurs, traitement logique et retour visuel pour l'utilisateur, tout en minimisant les besoins en ressources humaines.

Ainsi, l'utilisation de ce matériel, à la fois fiable, économique et facilement intégrable, permet de concevoir un système intelligent performant répondant efficacement aux besoins réels de gestion de stationnement ; et ouvre la voie à des améliorations futures telles que l'intégration de la connectivité IoT, d'applications mobiles ou encore d'algorithmes d'optimisation pour une gestion encore plus autonome et intelligente.